Variabler

Når en variabel(b) settes lik en annen(a) blir den en referanse til den verdien den andre variablen referer til. Om vi så endrer på variabelen(b) med å sette den lik noe annet (**dvs. endringer hvor man bruker et '=' tegn**) endrer det *ikke* på original variabelen.

```
a = 30

b = a   #b får verdien 30

b += 15 #b får verdien 45 (30+15)

print(a)#a har fortsatt verdien 30

print(b)#b har verdien 45
```

Om vi bruker **metoder som endrer verdier** derimot blir disse brukt på den variabelen(a) som variabelen(b) refererer til. Dette gjelder innebygde metoder og importerte metoder (fra libraries eller egendefinerte klasser.)

```
a = [4,3,2,1]

b = a   #b settes som referanse til a

b.sort()

print(a) #a er [1,2,3,4]

print(b) #b er referanse til a [1,2,3,4]
```

Metoder som **ikke endrer av seg selv**, AKA de har returverdi endrer kun der de settes '='.

```
a = "a b c d"

b = a   #b settes som referanse til a

b = b.split(' ') #b settes lik b.split(' ') som er a.split(' ')

print(a) #a er "a b c d"

print(b) #b er ["a","b","c","d"] aka a.split(' ')
```

Det vil si at kun metoder som direkte endrer på en variabel(b) endrer på variabelen(a) som den er en referanse til. Funksjoner kan ikke gjøre dette fordi de kun gjør endringer på lokale variabler. Siden metoder som .split() som har retur-verdi og ikke gjør endringer direkte vil derfor ikke endre variabelen som refereres til. (b) endres, men ikke (a).

I egendefinerte klasser vil metoder som endrer på instansvariabler endre på variabel(a) når det kalles på variabel(b) som er satt til å referere til variabel(a) som er satt lik et klasseobjekt. **Klasseobjekter vil endre alle variabler som refererer til det fordi det kun er et objekt som de alle referer til.**

NB: Bruk blyant og kladdeark når det gjelder variabler som referes fram og tilbake på kryss og tvers til den ene og den andre.

```
person1 = Person(10)
person2 = Person(20)

person3 = person2

person2 = person1

print(person2.hent_alder()) #person2 referer til Person(10)
print(person3.hent_alder()) #person3 referer fortsatt til Person(20)
```

```
a = 10

b = a

a += 10

print(a) #har verdi 20
print(b) #har fortsatt verdi 10
```

String formattering (kommer forhåpentligvis ikke på eksamen)

Ønsket ouput:

```
Rektangelet er 10.1 cm langt og 3.8 cm bredt.
```

Løsninger:

```
lengde = 10.101
bredde = 3.843
print("Rektangelet er %.1f cm langt og %.1f cm bredt." % (lengde, bredde))
#eller
print("Rektangelet er {0:.1f} cm langt og {1:.1f} cm bredt.".format(lengde,
bredde))
#eller
print(f"Rektangelet er {lengde:.1f} cm langt og {bredde:.1f} cm bredt.")
```

Ønsket output (gitt variabler diameter, areal og omkrets):

```
Diameter:    12.00
Areal:      113.10
Omkrets:     37.70
```

Løsninger:

```
# Gammel versjon
print("Diameter: % 10.2f" % diameter)
print("Areal: % 13.2f"  % areal)
print("Omkrets: % 11.2f" % omkrets)


# Nyere versjon: f-strings (t.o.m Python 3.6)
print(f"Diameter: {diameter:10.2f}")
print(f"Areal: {areal:13.2f}")
print(f"Omkrets: {omkrets:11.2f}")
```

Streng Metoder

Metoder (retur)	Funksjon (= indikerer returverdi)	Beskrivelse
.strip(streng)	‘123abc123’.strip(‘123’) = abc ‘123abc123’.strip(‘231’) = abc ‘123abc123’.strip(‘321’) = abc Passer bra til å fjerne linebreaks fra slutten av linjer. .strip(‘\n’)	Returnerer en streng hvor tegnene gitt som parameter er fjernet fra starten og slutten av strengen. Rekkefølgen på parameter strengen har ikke betydning. **.rstrip** for å kun fjerne fra slutten. **.lstrip** for å kun fjerne fra starten.
.split(streng, integer)	‘1 2 3 4 5’.split(‘ ’) = [‘1’,’2’,’3’, ‘4’, ‘5’] ‘1 2 3 4 5’.split(‘ ’, 2) = [‘1’,’2’,’3 4 5’] ‘1r2r3r3r4r5’.split(‘rdf’) = [‘1r2r3r4r5’]	Returnerer en liste hvor strengen er delt hvor parameterstrengen man oppgir finnes i strengen. Tom parameterstreng deler strengen i individuelle tegn. Mellomrom innkapslet i apostrofer deler etter mellomrom. Integer-parameteren er antall delinger som skal gjøres. Default er 0. Delinger telles fra venstre. **.rsplit()** kan brukes om man vil telle delinger fra høyre.

Note: All string methods returns new values. They do not change the original string.

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case. More aggressive than lower()
center()	Returns a centered string (length, character*) *space by default
count()	Returns the number of times a specified value occurs in a string (value, start*, end**) *0 by default **end of string by default

find()	Searches the string for a specified value and returns the position of where it was found (first occurrence) (value, start*, end**) *0 by default **end of string by default Returnerer "-1" om substring ikke er i søkefeltet til metoden.
index()	Searches the string for a specified value and returns the position of where it was found (first occurrence) (value, start*, end**) *0 by default **end of string by default "ValueError: substring not found" om den ikke er i søkefeltet til metoden.
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isascii()	Returns True if all characters in the string are ascii characters
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric

isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
isupper()	Returns True if all characters in the string are upper case
join()	Converts the elements of an iterable into a string string.join(iterable) Ex. : “hei”.join([“1”,”2”,”3”,”4”]) #elementer må være strings = : “1hei2hei3hei4”
ljust()	Returns a left justified version of the string string.ljust(length, character*) *Space by default.
lower()	Converts a string into lower case
replace()	Returns a string where a specified value is replaced with a specified value string.replace(oldvalue, newvalue, count*) *All occurrences by default.
rfind()	Searches the string for a specified value and returns the last position of where it was found

rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list
splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case

List methods

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position iterable.insert(pos, elmnt)
pop()	Removes the element at the specified position iterable.pop(index*) *-1 by default
remove()	Removes the first item with the specified value

reverse()	Reverses the order of the list
sort()	Sorts the list

Klasser:

Eksempel klassekode:

```
class Gjenstand:
    def __init__(self, navn):
        self._navn = navn
        self.beskrivelse = ''
        self.verdi = self.antall()

    def leggTilBeskrivelse(self, beskrivelse):
        self.beskrivelse = beskrivelse

    def returnerBeskrivelse(self):
        return self.beskrivelse

    def antall(self):
        return len(self._navn) + len(self.beskrivelse)

    def returnerVerdi(self):
        return self.verdi
```

Eksempel driverkode:

```
hammer = Gjenstand("hammer")
print(hammer.returnerVerdi()) #printer 6
hammer.leggTilBeskrivelse("Metallstykke med treskaft")
print(hammer.returnerBeskrivelse()) #printer "Metallstykke med treskaft"
print(hammer.returnerVerdi()) #fortsatt 6 fordi .verdi ble gitt før beskrivelse.
```

Om klassen er i egen fil må man importere:

```
from punkt import Punkt #from filnavn import klassenavn
```

Viktige Dunder metoder (__metoder__) i klassen din

def __init__(self, parameter1, parameter2, …)

I denne kan du definere instansvariabler. Variabler som tilhører det spesifikke klasseobjektet du håndterer. Variabler utenfor denne, klassevariabler, vil ha samme verdi for alle objekter av klassen.

def __str__(self):

Retur verdien til denne blir det som gis når print, str eller andre metoder som ber klasseobjektet oppføre seg som en streng kalles på et klasseobjekt.

Underscore? Grensesnitt?

En understrek før en variabel, funksjon, metode etc. indikerer at den kun er ment for bruk inne i klassen eller hva enn modul eller pakke den tilhører. Dette er en navngivningskonvensjon som ikke håndheves av python, men som indikerer hva som skal være internt. Om **metoden** eller **instansvariabelen *ikke*** har underscore foran er det en del av **grensesnittet (public interface)** til klassen. Om den har det er det en indikasjon på at den skal være **innkapslet (non-public)**.

If statement:

If statements slår ut om blokken etter evalueres til True. Eksempler:

```
if False:
    print("slår ikke ut")
if False == False:
    print("slår ut")
if True and False:
    print("slår ikke ut")
if True and False == False:
    print("slår ut")
if False and True:
    print("slår ikke ut")
if 1==1 or False == True:
    print("slår ut")
if not True:
    print("slår ikke ut")
if not False:
    print("slår ut")
if False ^ True: #xor
    print("slår ut")
```

Se på alt mellom if og : som en blokk og evaluer om den er True eller False. Som du kan se i skjermbilde blir statements som aldri er True grået ut av kode-editoren.

Alle strenger og tall (unntatt 0 og tom string) evalueres til True.

Samlinger

4 hovedtyper: list, tuple, set, dictionary.

List: [item1,item2,item3] // Konstruktør: list((item1,item2,item3))
-Indeksert (fast rekkefølge)
-Kan ha duplikater
-Gjenstander kan endres, fjernes og legges til
-Metoder: Se «List methods»

Tuple: (item1,item2,item3) // Konstruktør: tuple((item1,item2,item3))
-Indeksert
-Kan ha duplikater
-Kan ikke endres etter den er laget
-Metoder: Kun count() og index()

Set: {item1,item2,item3} // Konstruktør: set((item1,item2,item3))
-Ikke indeksert
-Kan ikke ha duplikater
-Kan ikke endre items, men kan fjerne og legge til.
-Metoder: add() som legger til elementet man gir som parameter, remove() som fjerner elementet man gir som parameter, pop() som fjerner et tilfeldig element.

Dictionary: {key1:value1, key2:value2, key3:value3} // Konstruktør: dict(key1 = value1, key2 = value2)
-Ordered
-Ikke duplikate nøkler.
-Kan endre, fjerne og legge til items (nøkkel:verdi par)
-Metoder: keys() som gir liste med keys, values() som gir liste med values, pop() som fjerner elementet med den nøkkelen man gir som parameter, get(keyname) som returnerer verien til nøkkelen gitt som parameter.